

Bisection Method

🔗 WHY THIS METHOD EXISTS

The goal of the bisection method is to **find a root** of a function i.e. find an x such that

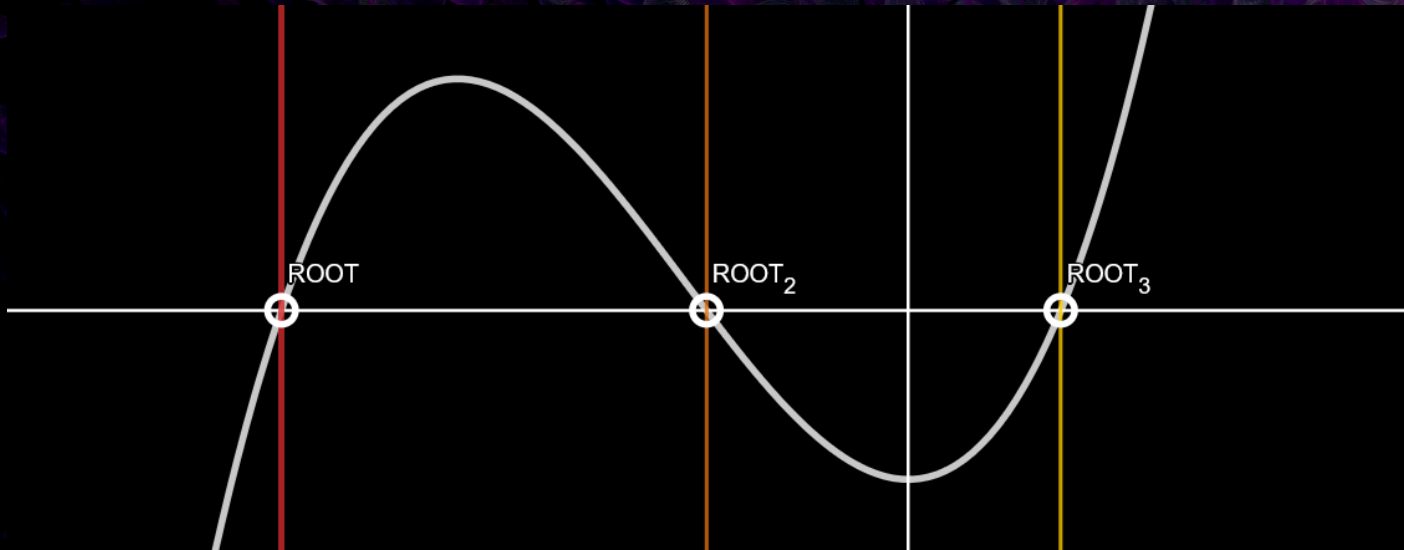
$$f(x) = 0.$$

Introduction

✍ ROOT OF A FUNCTION

A **root** of a function is a special value of the independent variable (x -value) such that the functional value (y -value) is zero.

Or simply put, $f(x) = 0$.

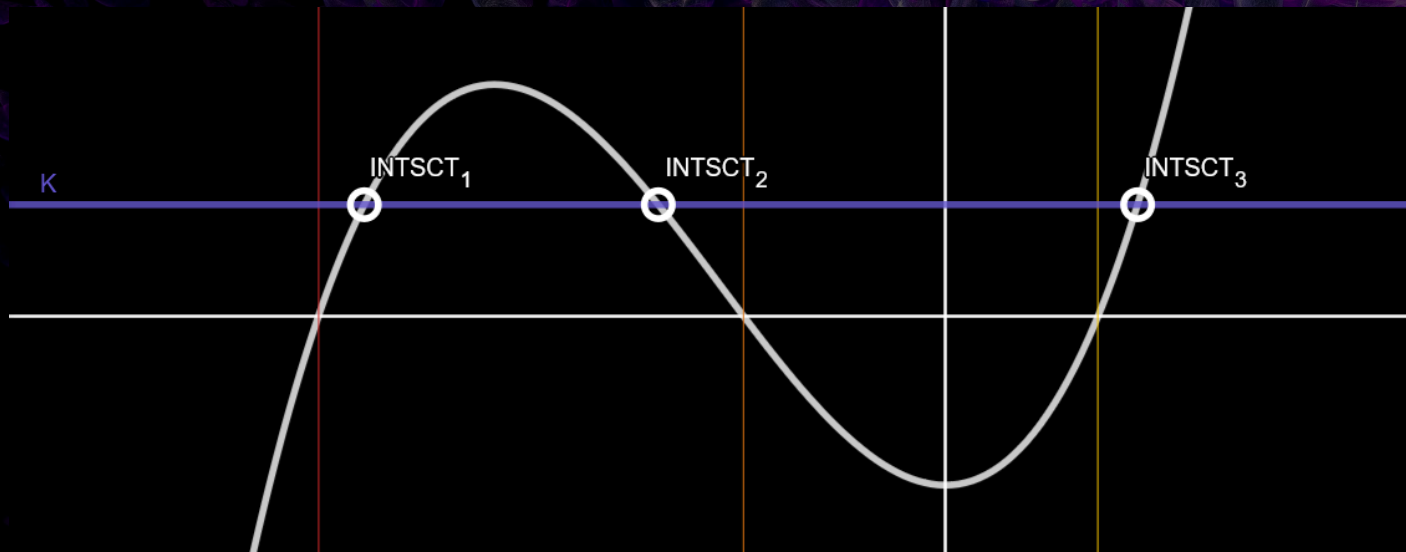


You already know that, when $f(x) = 0$, there exists a root at the point x .

❗ NOT JUST ZERO, ANY HORIZONTAL LINE

$f(x)$ doesn't have to be 0. It could be some value K .

For $f(x) = K$, we define $g(x) = f(x) - K$, and then $g(x) = 0$ same problem, shifted up/down by K .



Now, $f(x) = K$ at different points x .

Intermediate Value Theorem

✍ INTERMEDIATE VALUE THEOREM (IVT)

If f is **continuous** on $[a, b]$ and K is between $f(a)$ and $f(b)$, then there exists some $c \in [a, b]$ with $f(c) = K$.

✍ SPECIAL CASE: $K = 0$

If $K = 0$, then c is a **root** of the function. Equivalently, when $K = 0$ the functional values $f(a)$ and $f(b)$ have **opposite signs** (or one of them is exactly zero).

What IVT actually requires

For the guarantee to hold, two conditions must be satisfied:

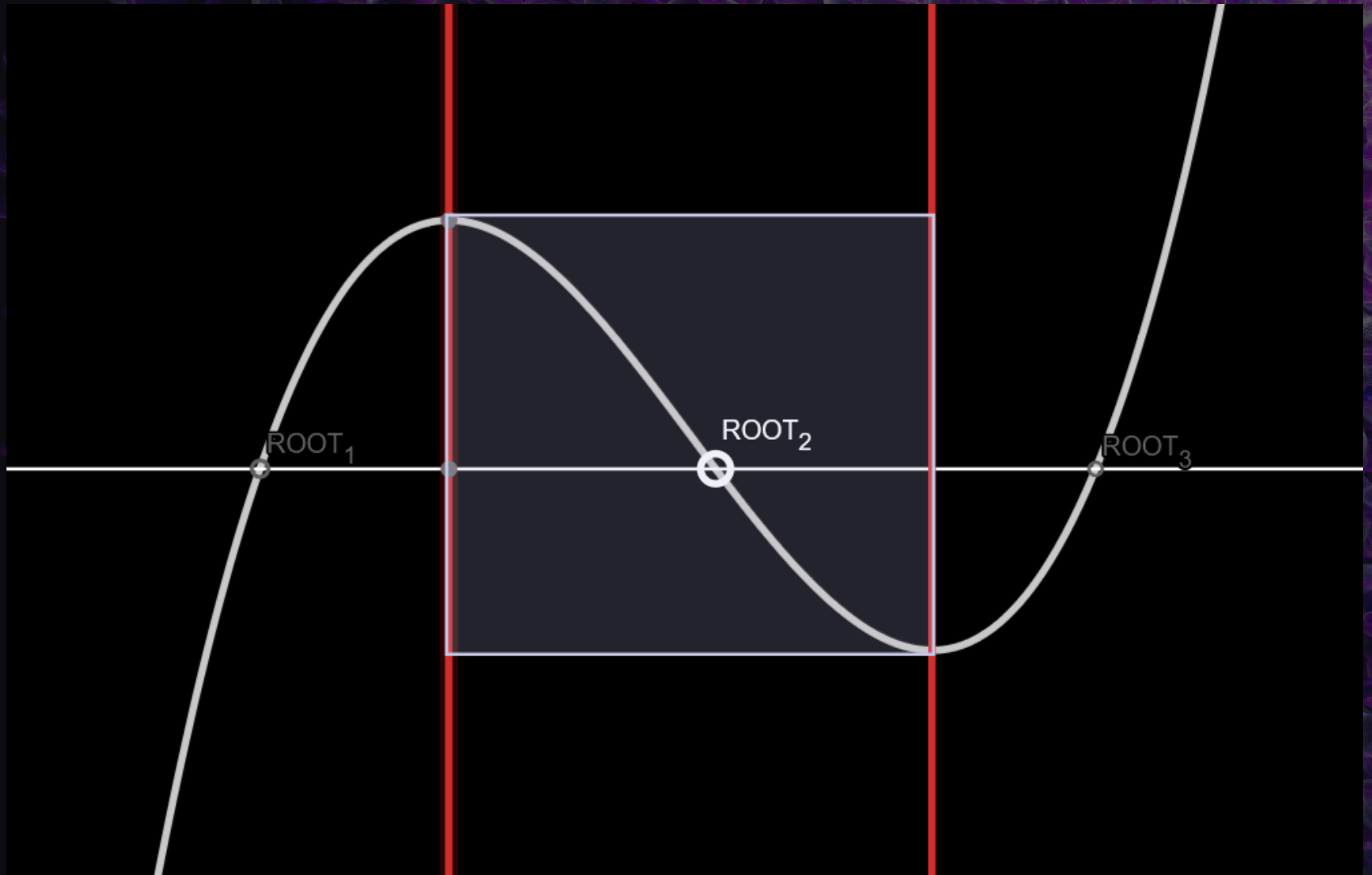
- The function must be **defined** on $[a, b]$, so every x -value in the interval must be assigned a y -value.
- The function must be **continuous** on $[a, b]$, so every intermediate y -value between $f(a)$ and $f(b)$ is actually attained.

✍ WHY THIS GUARANTEES A ROOT

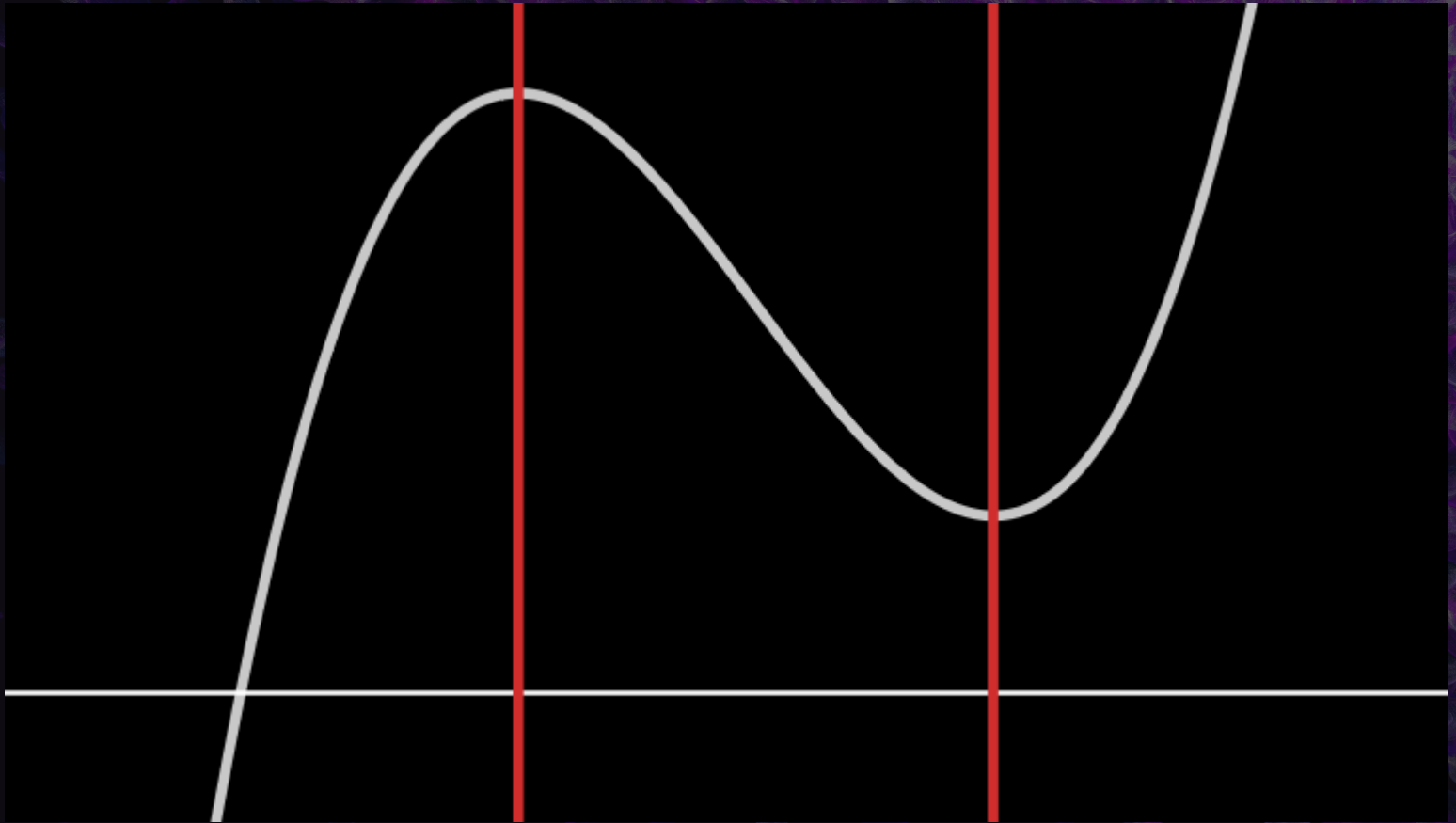
The two conditions together guarantee that **every y -value between $f(a)$ and $f(b)$ has an assigned x -value**. So if K sits between them, an x must exist with $f(x) = K$.

The position of K matters, here's why...

✓ K BETWEEN $F(A)$ AND $F(B)$ → GUARANTEED INTERSECTION



✕ K OUTSIDE $F(A)$ AND $F(B)$ → NO GUARANTEE



 SUMMARY

If K lies outside $[f(a), f(b)]$, there is no guarantee of an intersection. The function might cross the line, or might not.

Bisection Method

CORE IDEA

Whenever the functional values of f cross a **root** (x -axis, $y = 0$), they **change sign** or one of them is exactly 0.

Step-by-step

1. **Find a sign change.** Locate an interval $[a, b]$ where $f(a) \cdot f(b) < 0$.

2. **Split it in half.** Compute the midpoint

$$c = \frac{a + b}{2}.$$

3. **Pick the half that still changes sign.** Either $f(a) \cdot f(c) < 0$ **or** $f(c) \cdot f(b) < 0$

The product over the full interval $[a, b]$ is negative, so the negative product lives in exactly one of the two halves.

4. **Rename endpoints.** Replace c with whichever of a or b keeps the sign change.

For example, if $f(a) \cdot f(c) < 0$, replace $c \mapsto b$ and you once again have $f(a) \cdot f(b) < 0$.

5. **Repeat** until a stopping condition is met.

WHY THE INTERVAL ALWAYS SHRINKS

Each step halves $[a, b]$. After n steps the interval has width $\frac{b_0 - a_0}{2^n}$ which is exactly what makes the method **converge**.

Stopping conditions (Convergence Criteria)

There are two common ways to stop:

✓ EXACT HIT

$f(c) = 0$: the root is found **exactly** (rare in floating point).

✓ TOLERANCE ON WIDTH

$|b - a| < \varepsilon$: the interval is small enough that any point inside is a good approximation.

How many iterations do I need?

To **guarantee** $|b - a| < \varepsilon$ starting from $[a_0, b_0]$:

$$n_{\min} = \log_2 \left(\frac{b_0 - a_0}{\varepsilon} \right)$$

i HALVING INTUITION

Halving the interval n times gives width $\frac{b_0 - a_0}{2^n}$. Solve for the n that drops the width below ε and this is what you get.

How to code it

The script has **four moving parts**: input parsing, continuity check, bisection loop, then output. Each one is small.

HOUSEKEEPING

Clears the console, wipes workspace variables, and forces `long` so iteration output meets the 6 decimal requirement.

```
clc; clear; format long;
```

INPUT

*Take the function **as a string** in terms of `x`, plus the lower and upper limits of the interval.*

```
fx_string, lower, upper → from user
```

MINOR CALCULATIONS

Turn the user-typed string into a MATLAB function `fx(x)` that can actually be called.

```
string → callable handle
```

CONTINUITY

Core concept: **sample, slide across, compare neighbours with a tolerance:**

*Sample the interval at N slices, evaluate f_x at each, and check whether any **jumps** appear. If yes, throw an error.*

fill x s across the interval (100-300 points is good enough!)

fill f_x s by evaluating f_x at each x

for each interior point

ask: does this point sit roughly halfway between its two neighbours?

if the deviation is bigger than a tolerance

→ mark as discontinuous

if marked

error and exit

WHY THE MIDPOINT CHECK WORKS

If f_x were continuous, the value at $x_values(j)$ should sit roughly halfway between its neighbours. A deviation bigger than tol_jump means the curve jumped and IVT no longer applies.

BISECTION

Core concept: a for-loop with two `if` s inside:

The actual method, repeated until the interval is small enough or `fx(mid)` is 0.

check the bracket: do the endpoints change sign?

if not → exit, IVT does not apply

loop up to `max_iterations`

split the interval in half at `mid`

if we've shrunk enough, or `fx(mid)` is tiny enough → stop

if the sign change is on the left half → keep left

else → keep right

🔥 WHY THIS IS THE ALGORITHM

`mid` halves the bracket. The first `if` exits when we're close enough. The second `if / else` keeps the half that still changes sign, so that the next iteration starts with a fresh sign change across the bracket.

OUTPUT

Plot the function over the interval with axis labels, then report the result.

plot the curve

print the root, $f(\text{root})$, and iteration count

`root` is just the **midpoint of the final interval** the bracket is guaranteed to straddle the root, so its middle is the best estimate we have.

⚠ FLOATING-POINT PITFALLS (MATLAB)

- **Never compare floats with `==`**. `fc == 0` is almost never true. Use `abs(fc) < tol_f` a *function-value* tolerance alongside the width tolerance.
- **Guard against `Inf` / `NaN`**. If `f` grows large near the root, `f(c)` may return `Inf` or `NaN`. Wrap with `isfinite(fc)` before the sign-change test.
- **Width tolerance, not step tolerance.** Bound `abs(b - a) < tol` that's the real measure of how far apart your two guesses are.
- **Cap the loop.** Always set a `maxIter` limit.

Pros and Cons

✓ ADVANTAGES

- A root is **guaranteed** whenever the IVT preconditions are met.
- Known number of iterations.
- **No derivative** is required!

✗ DISADVANTAGES

- The method converges **slowly (linearly)** compared to Newton Raphson or Secant.
- If f is very **flat** near the root, it can take many iterations to bring $|b - a|$ below tolerance ε even when the interval is already small.
- The method **cannot find a root exactly at a turning point**. If the curve just *touches* the x -axis without crossing, $f(a) \cdot f(b) \geq 0$ and there is no sign change to exploit.

Convergence Bound

❓ WHERE DOES $N_{\min} = \log_2\left(\frac{B-A}{\epsilon}\right)$ COME FROM?

After each iteration the interval width is halved:

$$w_n = \frac{b_0 - a_0}{2^n}.$$

We want the smallest integer n such that $w_n < \epsilon$:

$$\frac{b_0 - a_0}{2^n} < \epsilon \iff 2^n > \frac{b_0 - a_0}{\epsilon} \iff n > \log_2\left(\frac{b_0 - a_0}{\epsilon}\right).$$

So the minimum number of iterations that **guarantees** $|b - a| < \epsilon$ is

$$n_{\min} = \left\lceil \log_2\left(\frac{b_0 - a_0}{\epsilon}\right) \right\rceil$$

🔗 WHY THIS IS A GUARANTEE

The bound doesn't depend on f at all. only on the starting width and the tolerance. That's the trade-off: slow but assured.

Examples / Tutorial Questions

Example 1: $\sqrt{x} - \cos(x)$ on $[0, 1]$, find c_3 (third approximation to the root)

Setup. $f(x) = \sqrt{x} - \cos(x)$, interval $[0, 1]$, sign-change bracket.

$$f(0) = 0 - 1 = -1 < 0$$

$$f(1) = 1 - \cos(1) \approx 1 - 0.5403 = 0.4597 > 0 \checkmark \text{ bracket valid.}$$

n	a_n	b_n	$c_n = \frac{a_n + b_n}{2}$	$f(c_n)$	sign of $f(a_n) \cdot f(c_n)$	new bracket
1	0	1	0.5000	$\sqrt{0.5} - \cos(0.5) \approx -0.1705$	+	$[0.5, 1]$
2	0.5	1	0.7500	$\sqrt{0.75} - \cos(0.75) \approx +0.1343$	-	$[0.5, 0.75]$
3	0.5	0.75	0.6250	$\sqrt{0.625} - \cos(0.625) \approx -0.0194$	-	$[0.625, 0.75]$

$$c_3 = 0.625$$

Example 2: $x^3 - 7x^2 + 14x - 6 = 0$ on $[0, 1]$, accurate to 10^{-2}

Setup. $f(x) = x^3 - 7x^2 + 14x - 6$, interval $[0, 1]$.
 $f(0) = -6 < 0$, $f(1) = 1 - 7 + 14 - 6 = +2 > 0$ ✓ *bracket valid*.
Required iterations: $n_{\min} = \log_2\left(\frac{1-0}{10^{-2}}\right) = \log_2(100) \approx 6.64 \Rightarrow n_{\min} = 7$.

n	a_n	b_n	c_n	$f(c_n)$	sign change on	new bracket	width
1	0	1	0.5000	-0.6250	left $[a_n, c_n]$	$[0.5, 1]$	0.5
2	0.5	1	0.7500	+0.9844	right $[c_n, b_n]$	$[0.5, 0.75]$	0.25
3	0.5	0.75	0.6250	+0.2598	right $[c_n, b_n]$	$[0.5, 0.625]$	0.125
4	0.5	0.625	0.5625	-0.1619	left $[a_n, c_n]$	$[0.5625, 0.625]$	0.0625
5	0.5625	0.625	0.59375	+0.0541	right $[c_n, b_n]$	$[0.5625, 0.59375]$	0.03125
6	0.5625	0.59375	0.578125	-0.0526	right $[c_n, b_n]$	$[0.578125, 0.59375]$	0.015625
7	0.578125	0.59375	0.5859	$\approx +0.0011$	right $[c_n, b_n]$	$[0.578125, 0.5859]$	0.0078

$$c \approx \frac{0.578125 + 0.5859}{2} \approx 0.5820$$

THE OTHER INTERVALS (2B, 2C) SAME RECIPE

Repeat the same procedure by yourself on the next two intervals, $[1, 3.2]$ and $[3.2, 4]$.

Example 3: $2 + \cos(e^x - 2) - e^x = 0$ on $[0.5, 1.5]$, accurate to 10^{-3}

Setup. $f(0.5) \approx +1.288$, $f(1.5) \approx -3.269$ ✓ bracket valid.
Required: $n_{\min} = \log_2\left(\frac{1}{10^{-3}}\right) = \log_2(1000) \approx 9.97 \Rightarrow n_{\min} = 10$.

n	a_n	b_n	c_n	$f(c_n)$	sign change on	new bracket	width
1	0.5	1.5	1.0000	+0.0339	$[c_n, b_n]$	$[1, 1.5]$	0.5
2	1	1.5	1.2500	-1.4108	$[a_n, c_n]$	$[1, 1.25]$	0.25
3	1	1.25	1.1250	-0.6106	$[a_n, c_n]$	$[1, 1.125]$	0.125
4	1	1.125	1.0625	-0.2658	$[a_n, c_n]$	$[1, 1.0625]$	0.0625
5	1	1.0625	1.03125	-0.1104	$[a_n, c_n]$	$[1, 1.03125]$	0.03125
6	1	1.03125	1.01563	-0.0366	$[a_n, c_n]$	$[1, 1.015625]$	0.015625
7	1	1.015625	1.00781	-0.0011	$[a_n, c_n]$	$[1, 1.007813]$	0.007813
8	1	1.007813	1.00391	+0.0164	$[c_n, b_n]$	$[1.003906, 1.007813]$	0.003906
9	1.003906	1.007813	1.00586	+0.0077	$[c_n, b_n]$	$[1.005859, 1.007813]$	0.001953
10	1.005859	1.007813	1.006836	+0.0033	$[c_n, b_n]$	$[1.005859, 1.006836]$	0.000977

$c \approx \frac{1.005859 + 1.006836}{2} \approx 1.0063$

Example 4: $f(x) = (x + 2)(x + 1)x(x - 2)(x - 1)^3$: which zero does it converge to?

Think about this, and try to work it out yourself.

Setup. f has zeros at $x = -2, -1, 0, 1$ (triple), 2 .

4a: $[1.5, 2.5]$. $f(1.5)$ and $f(2.5)$ have **opposite signs**, so the bracket contains exactly one sign change at $x = 2$.

Bisection converges to $\boxed{x = 2}$.

Try these yourself!

4b: $[-0.5, 2.4]$.

4c: $[-1.75, 1.5]$.

Example 5: Approximate $\sqrt{3}$ to within 10^{-4}

Setup. Want $\sqrt{3}$, so solve $f(x) = x^2 - 3 = 0$ on $[1, 2]$.

$f(1) = 1 - 3 = -2$, $f(2) = 4 - 3 = +1 \rightarrow$ sign change, bracket valid.

Required: $n_{\min} = \log_2 \left(\frac{2 - 1}{10^{-4}} \right) = \log_2(10^4) \approx 13.29 \Rightarrow n_{\min} = 14$.

n	a_n	b_n	c_n	c_n^2	sign change on	new bracket	width
1	1	2	1.5000	2.2500	$[c_n, b_n]$	$[1.5, 2]$	0.5
2	1.5	2	1.7500	3.0625	$[a_n, c_n]$	$[1.5, 1.75]$	0.25
3	1.5	1.75	1.6250	2.640625	$[c_n, b_n]$	$[1.625, 1.75]$	0.125
4	1.625	1.75	1.6875	2.847656	$[c_n, b_n]$	$[1.6875, 1.75]$	0.0625
5	1.6875	1.75	1.71875	2.954102	$[c_n, b_n]$	$[1.71875, 1.75]$	0.03125
6	1.71875	1.75	1.734375	3.008057	$[a_n, c_n]$	$[1.71875, 1.734375]$	0.015625
7	1.71875	1.734375	1.726563	2.980980	$[c_n, b_n]$	$[1.726563, 1.734375]$	0.007813
8	1.726563	1.734375	1.730469	2.994522	$[c_n, b_n]$	$[1.730469, 1.734375]$	0.003906
9	1.730469	1.734375	1.732422	3.001284	$[a_n, c_n]$	$[1.730469, 1.732422]$	0.001953
10	1.730469	1.732422	1.731445	2.997903	$[c_n, b_n]$	$[1.731445, 1.732422]$	0.000977
11	1.731445	1.732422	1.731934	2.999593	$[c_n, b_n]$	$[1.731934, 1.732422]$	0.000488
12	1.731934	1.732422	1.732178	3.000439	$[a_n, c_n]$	$[1.731934, 1.732178]$	0.000244
13	1.731934	1.732178	1.732056	3.000016	$[a_n, c_n]$	$[1.731934, 1.732056]$	0.000122
14	1.731934	1.732056	1.731995	2.999804	$[c_n, b_n]$	$[1.731995, 1.732056]$	6.1×10^{-5}

True value: $\sqrt{3} \approx 1.7320508$ within 10^{-4} . ✓

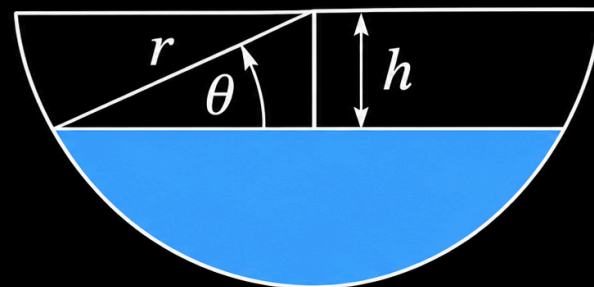
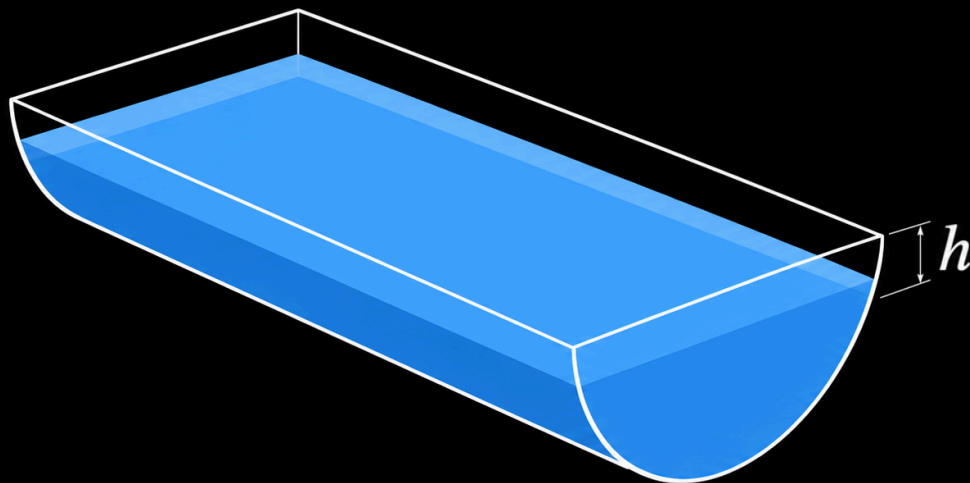
Example 6: Approximate $\sqrt[3]{25}$ to within 10^{-4} (no worked example)

Example 7: Trough volume (challenge, no worked example)

Setup. A (semicircular) trough of length L has a cross section in the shape of a semicircle, with radius r , when filled with water to a distance h from the top, the volume V of water is

$$V = L[0.5\pi r^2 - r^2 \arcsin(\frac{h}{r}) - h(r^2 - h^2)^{1/2}]$$

Suppose $L = 10\text{ft}$, $r = 1\text{ft}$, $V = 12.4\text{ft}^2$. Find the **depth** of the water to **within** 0.01ft .



PS: make sure to check the module guide's textbooks for similar questions!